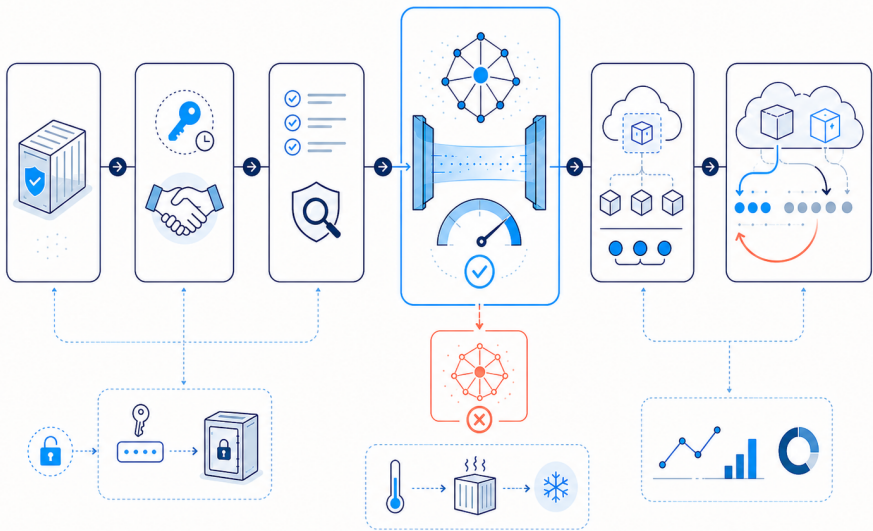


CI/CD For AI Projects



Normal CI/CD asks "does the code run?" AI CI/CD also asks **"is the model still right?"** That second question is the stage most teams skip.



AI CI/CD Is Not Normal CI/CD, The Output Is Non-Deterministic

PROBLEM

You cannot assert the answer equals a fixed string. The model gives a different, valid answer every time.

= NORMAL CI TEST

```
assert result == "30 days"
```

exact match, deterministic,
pass/fail — **and it flakes on an LLM.**

✓ AI CI TEST

```
assert "30 days" in answer  
assert cites(policy_doc)  
assert faithfulness > 0.8
```

a score against a threshold



Assert **properties, not equality**: contains the right fact, cites a real source, valid structure, scores above a bar. An exact-match test on an LLM either flakes or tests nothing.



AI Coach John

FOLLOW



REPOST

It Is Not "Just A Docker Container"

CORRECTION

The container is the easy 20%. The AI 80% is the eval gate, the secrets, the cold start, and tests that survive non-determinism.



THE TUTORIAL COVERS

write a Dockerfile
push to a registry
gcloud run deploy
"it's live"



WHAT ACTUALLY MATTERS

- + an eval gate (quality)
- + secrets never in the image
- + cold-start mitigation
- + tests that handle non-determinism



The Cloud Run mechanics are real, but they are the part you can copy from any tutorial. **The AI-specific stages decide whether your deployment is a demo or a product.**



AI Coach John

FOLLOW



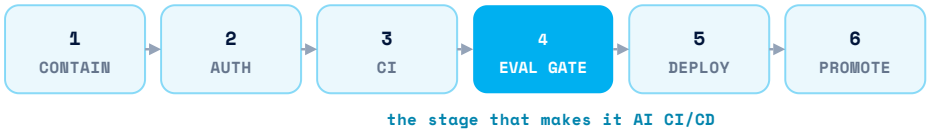
REPOST

The Pipeline, Six Stages

OVERVIEW

Every push to main runs the same six. A failure at any stage stops the deploy.

EVERY PUSH TO MAIN RUNS ALL SIX · A FAILURE AT ANY STAGE STOPS THE DEPLOY



1 Containerize. A Dockerfile that builds the app and its deps.

2 Authenticate. GitHub Actions to GCP, via Workload Identity.

3 CI. Lint, unit test, security scan, build, push the image.

4 Eval gate. Run the golden dataset, fail if quality drops.

5 Deploy. Push the image to Cloud Run, staging first.

6 Promote. Integration-test staging, then shift prod traffic.



Slim The Image, Keep Data Out Of It

CONTAINERIZE

Two traps: a bloated image that is slow to build and deploy, and a model or index baked in that you cannot update without a rebuild.

SLIM BASE, SERVING DEPS ONLY, NO DATA BAKED IN

```
FROM python:3.12-slim                # not a full CUDA image
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY ./app ./app
# do NOT COPY the vector index or model weights in here
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8080"]
```

⚠ TWO TRAPS

FAT IMAGE

full CUDA + every lib → multi-GB, slow build, slow cold start.
Fix: slim base, serving deps only.

DATA BAKED IN

index in the image → a content change needs a full rebuild.
Fix: read it at runtime from a store.



Cloud Run wants your app on \$PORT (8080) and starting fast. The slim build is not tidiness — **it is a deploy-time and cold-start requirement.**



AI Coach John

FOLLOW



REPOST

Workload Identity, Not A JSON Key

AUTHENTICATE

The tutorial says download a service-account key and paste it into a secret. That is a long-lived credential in a file, and it is wrong.

GITHUB ACTIONS: KEYLESS AUTH TO GCP

```
- uses: google-github-actions/auth@v2
  with:
    workload_identity_provider: projects/123/.../providers/gh
    service_account: deployer@my-project.iam.gserviceaccount.com
    # note: NO credentials_json, NO key file
```

JSON KEY (WRONG)

a key file in a GitHub secret · never expires · **a leak is standing access**

WORKLOAD IDENTITY

no key file at all · a short-lived token per run · **nothing to rotate or steal**



Prefer a **short-lived, scoped credential over a long-lived secret.**

Configure the trust once; the pipeline then authenticates with no stored key.



AI Coach John

FOLLOW

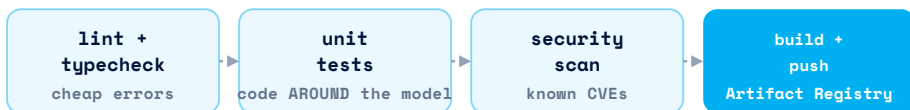


 REPOST

The Standard Gates, **Non-Negotiable**

CI

Before anything AI-specific: lint, unit tests, a security scan, build the image, push to Artifact Registry.



WHAT THESE COVER

Unit tests cover the **deterministic logic**: retrieval plumbing, parsing, the API layer — everything around the model that is normal code. The AI-specific stages assume this is sound.

Tag the image with the **git SHA**, not **latest**. Every deploy traces to one commit, and a rollback is just redeploying an earlier SHA.



AI Coach John

FOLLOW



REPOST

The Eval Gate

THE AI STAGE

Run the app against a golden dataset, score the quality, and fail the build if it dropped below the bar. A prompt change cannot sneak past this.

RUNS IN CI AFTER THE BUILD – FAILS THE JOB IF QUALITY DROPS

```
def eval_gate(app, golden_set, min_score=0.85):
    scores = []
    for q, gold in golden_set:
        answer = app.query(q)
        scores.append(grade(answer, gold))    # metric or LLM judge
    avg = sum(scores) / len(scores)
    assert avg >= min_score, f"QUALITY REGRESSION: {avg:.2f}"
```

CHANGE	UNIT + BUILD	EVAL GATE
tweak the prompt	pass	0.71 FAIL
bump the model	pass	0.79 FAIL
refactor parsing	pass	0.88 pass



Keep the golden set small and real, grow it from every production failure, and set the bar **just below your current score** so a regression fails but normal variance does not.



Deploy To Cloud Run, Staging First

DEPLOY

One command ships the image. Ship it to a staging service first, never straight to the URL your users hit.

DEPLOY-CLOUDRUN ACTION, STAGING SERVICE

```
- uses: google-github-actions/deploy-cloudrun@v2
  with:
    service: support-agent-staging           # staging, not prod
    image: us-docker.pkg.dev/proj/repo/app:${{ github.sha }}
    region: us-central1
    flags: '--min-instances=1 --memory=2Gi --port=8080'
```

image (SHA) → Cloud Run STAGING — no users



integration tests run against it



tests pass? → Cloud Run PROD → users



Deploy straight to prod and **your users are the integration test**. Note - `-min-instances=1` even in staging — the flags are where the real decisions live.



Traffic Split And Instant Rollback

[PROMOTE](#)

Do not flip 100% to a new revision at once. Cloud Run keeps every revision, so shift gradually and roll back instantly.

TRAFFIC SPLIT WHILE YOU WATCH ERROR RATE AND LATENCY



the known-good revision keeps serving almost everyone

SHIFT TRAFFIC — AND ROLL BACK THE SAME WAY

```
gcloud run services update-traffic support-agent \  
  --to-revisions=NEW=10,PREVIOUS=90
```

```
# holds? --to-revisions=NEW=100  
# spikes? --to-revisions=PREVIOUS=100
```



The previous revision is **one command away, always**, still running. Rollback is a traffic shift — no rebuild, no redeploy.



AI Coach John

[FOLLOW](#)[REPOST](#)

Secrets, Never In The Image

BLIND SPOT

Your model API key is not an env var in the Dockerfile and not a value in the repo. It lives in Secret Manager.

⚠️ WRONG

ENV KEY=sk-... in the Dockerfile
hardcoded in the code
.env copied into the image
in the image = in the leak

✅ RIGHT

Secret Manager holds the key
Cloud Run injects it at runtime
--set-secrets on deploy
never in image, repo or logs

CLOUD RUN RESOLVES THE SECRET AT RUNTIME

```
flags: '--set-secrets=OPENAI_API_KEY=openai-key:latest'
```



Covers every credential: DB connection string, JWT secret, third-party token. **If it is sensitive, it is a runtime secret reference, not a value in your build.**



AI Coach John

FOLLOW



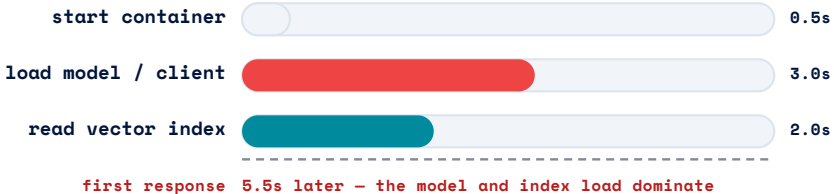
REPOST

Cold Starts, The AI-Specific Latency Killer

SILENT KILLER

A normal web app cold-starts in milliseconds. An AI app that loads a model or index on startup takes seconds, and Cloud Run scales to zero by default.

FIRST REQUEST, NO WARM INSTANCE



A CRUD app connects to a DB and is ready. An AI app **pulls a model, warms a client and loads an index** — all on the critical path of the first request after a scale-to-zero.



`--min-instances=1` keeps one instance warm, so there is **never a cold start on the first request**, at the cost of paying for that idle instance.



AI Coach John

FOLLOW



REPOST

Sizing Cloud Run For An AI Workload

CONFIGURATION

The defaults are tuned for a lightweight web service. An AI app needs more memory, careful concurrency, a longer timeout, sometimes a GPU.

SETTING	DEFAULT (WEB)	AN AI APP NEEDS
memory	small	GBs: model + index + headroom
concurrency	high	lower — LLM calls are heavy
request timeout	short	longer — slow or streaming generation
GPU	none	only if you HOST the model

 **OOM on start** = memory too low for the model + index. **Answer cut off** = request timeout shorter than the generation.

 An AI container is heavier and slower per request: **more memory, less concurrency, more time**. Reach for a GPU only if you host the model yourself.



Watch Quality, Not Just Uptime

AFTER IT SHIPS

Uptime and latency say the service is running. They do not say the answers went bad.

INFRA (CLOUD RUN GIVES YOU)

is it up?

latency, error rate

cold starts, cost

QUALITY (YOU ADD)

are the answers still good?

sample + score prod traffic

drift: fallbacks, length, guardrails

 Up and fast and **wrong** is invisible to infra monitoring alone. Run the same eval on sampled prod traffic, so a regression that slipped the gate surfaces as an **alert, not a complaint**.



AI Coach John

FOLLOW



 REPOST

The Full Workflow, One File

THE WHOLE THING

Every numbered stage is one or two steps. The eval gate sits in the middle, where a failing score stops everything after it.

.GITHUB/WORKFLOWS/DEPLOY.YML

```
name: deploy
on: { push: { branches: [main] } }
permissions: { id-token: write, contents: read } # keyless auth
jobs:
  ship:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: make lint test # stage 3: CI
      - uses: google-github-actions/auth@v2 # stage 2: keyless
        with: { workload_identity_provider: ..., service_account: ... }
      - run: gcloud builds submit --tag $IMG:${{ github.sha }} # 1+3
      - run: python eval_gate.py # stage 4: EVAL GATE
      - uses: google-github-actions/deploy-cloudrun@v2 # stage 5
        with:
          service: support-agent-staging
          image: $IMG:${{ github.sha }}
          flags: '--min-instances=1 --set-secrets=OPENAI_API_KEY=key:latest'
      - run: python integration_test.py --url $STAGING_URL # stage 6
      - run: gcloud run services update-traffic support-agent \
        --to-revisions=${{ github.sha }}=10 # gradual prod
```



Not aspirational — a real, small file. The AI-specific parts (the **eval gate**, **keyless auth**, **secret mount**, **min-instances flag**) are each a single line.



Scale-To-Zero Versus Always-Warm

THE TRADEOFF

Scale to zero and pay nothing idle, but eat a cold start on every wake. Keep one warm and pay for idle to kill it. There is no free setting.

MIN-INSTANCES=0

COST AT REST	~zero
COLD START	on every wake
FITS	sporadic, internal tools

MIN-INSTANCES=1

COST AT REST	one idle instance
COLD START	never
FITS	steady, user- facing



Set `min-instances` per service by how much a cold start costs your users versus how much an idle instance costs you.



Which Settings For Your App?

DECISION PATH

Three questions set the flags that matter, before you write the pipeline.

Q1

User-facing and latency-sensitive? Yes → `--min-instances=1`, no cold start, pay for warm. No → 0 is fine, accept a slow first hit.

Q2

Host the model, or call a hosted API? Host → attach a GPU and size memory to the model. Hosted → no GPU, size to the index + app.

Q3

Stream long generations? Yes → raise the request timeout well above the default. No → the default is fine.



Not optional, whatever you answered: **keyless auth**, **secrets in Secret Manager**, **images tagged by SHA**, and an **eval gate before every deploy**.



AI Coach John

FOLLOW



REPOST

What You See, Which Stage Is Behind It

DEBUG TABLE

The stages map onto the pipeline on slide 4.

SYMPTOM	CAUSE	STAGE
Green build, answers got worse	no eval gate	eval gate
First request each morning slow	scale-to-zero cold start	cold start
Deploy slow, image huge	fat image, deps + CUDA	containerize
API key leaked	key baked into the image	secrets
Container OOMs on startup	memory too low for the model	sizing
Long answers cut off	request timeout too short	sizing
Cannot roll back cleanly	tagged "latest", not by SHA	CI
Bad deploy took down all users	100% cutover, no split	promote
Updating docs needs a redeploy	index baked into the image	containerize



Nine Ways It Breaks, Nine One-Line Fixes

MISTAKES

Most of these are one flag or one stage. None of them are hard.

MISTAKE	FIX
No eval gate, ship on green	add the golden-dataset gate
Deploy straight to prod	staging first, then a traffic split
min-instances=0 on a user app	keep one instance warm
Secrets in the image or repo	Secret Manager, injected at runtime
JSON key in a GitHub secret	Workload Identity, keyless
Image tagged "latest"	tag by git SHA, roll back by SHA
Index baked into the container	read it at runtime from a store
Default memory and timeout	size for the model, raise the timeout
Monitoring uptime only	add quality monitoring on prod



Ship The Container, Gate The Quality

CHEATSHEET

Cloud Run is solved. Proving the model is still right before it reaches a user is not solved by a tutorial.

STAGE / CONCERN	THE ONE LINE THAT MATTERS
containerize	slim image, data read at runtime, not baked in
authenticate	Workload Identity, keyless, no JSON key
CI	lint + test + scan + build, tag by git SHA
eval gate	golden dataset; fail the build if quality drops
deploy	staging first, never straight to the prod URL
promote	gradual traffic split, instant rollback by revision
secrets	Secret Manager, injected at runtime, never in image
cold start	min-instances by whether users feel the wait
sizing	more memory, less concurrency, longer timeout
monitor	watch answer quality, not just uptime

 The container is the easy 20%. **The eval gate is the other 80%.** That is a deployment you can push to on a Friday.